

V2 change: yellow highlighted

Subscripto: Subscription Lifecycle Management and Shared Bill Splitting System

Wen Yu

Department of Computer Science, Monroe University, King Graduate School

26SP-CS625-152W - Object Oriented Software

Professor Shahaboddin Shamshirband

Jun 27, 2026

Summary

Modern daily life is filled with an overwhelming variety of digital subscription services ranging from streaming platforms and gaming passes to cloud storage and fitness memberships. Digital subscriptions often renew on different dates and are difficult to coordinate in shared households. This issue is magnified in shared households where roommates and family members frequently split bills or accidentally subscribe to identical services on the same platform. Subscripto is a Python-based software application designed to centralize subscription tracking, optimize household expenses, and prevent unnecessary financial waste through an intuitive local interface. It provides individual users and shared households with a unified dashboard to monitor active expenses, pause services temporarily, and transfer bill ownership seamlessly between members. By automating renewal alerts and dynamically calculating shared household bills, this application ensures that the user make payment on time and have a centralized platform to manage complicated payment.

Problem Statement and Objectives

Many consumers completely lose track of their active digital subscriptions and frequently forget when a recurring fee is about to be automatically charged to their accounts. This common oversight leads to unwanted auto-renewals that cause significant financial drain over time because people pay for services they no longer use. Additionally, coordinating shared subscription costs among roommates or family members often involves chaotic manual calculations, spreadsheet tracking, and awkward social reminders about money.

The primary objective of this project is to build a robust object-oriented software system that resolves these tracking and budget-sharing complications under a single interface. The software will allow comprehensive lifecycle tracking so that users can pause, delete, or transfer subscriptions instantly depending on their monthly needs. Another major objective is

to implement an active duplicate detection algorithm that immediately warns users if they attempt to add an identical service platform that someone else in the house is already paying for. Furthermore, the system aims to automate complex bill splitting calculations and provide strict input validation to ensure all financial shares are mathematically accurate. The renewal alert should identify subscriptions due within the next 24 hours, not only those exactly 24 hours away.

Revised Objective	Matching Software Contribution
Design a model for users, roommates, subscriptions, and ownership transfer.	A class-based architecture for subscription lifecycle management.
Implement duplicate detection, renewal alerts, and bill splitting.	A functional subscription workflow with validation and financial logic.
Add file storage, exception handling, and test cases.	A reliable local application with persistent records and tested calculations.

Background or Related Systems

Standard personal finance applications like Mint or Rocket Money allow users to view their bank transactions and identify recurring bills after they have occurred. However, these traditional systems are heavily limited because they operate strictly as passive post-payment trackers rather than active management tools. They cannot proactively prevent a duplicate subscription before the money leaves an account, nor do they offer native tools to manage shared split ratios for a single account among multiple roommates.

Besides that, shared-expense tools such as Splitwise support cost sharing, but they do not focus specifically on subscription lifecycle states, duplicate subscription prevention, ownership transfer, and renewal-date alerts. Subscripto extends the idea by these existing commercial examples by introducing active subscription lifecycle states and precise collaborative billing logic. Instead of just reading past bank statements, our application allows users to simulate pausing a service or dynamically transferring payment ownership to another household member before the next billing cycle hits. It bridges the gap between

individual budgeting and collaborative expense sharing in a way that standard banking applications currently do not support.

Object-Oriented Design Plan

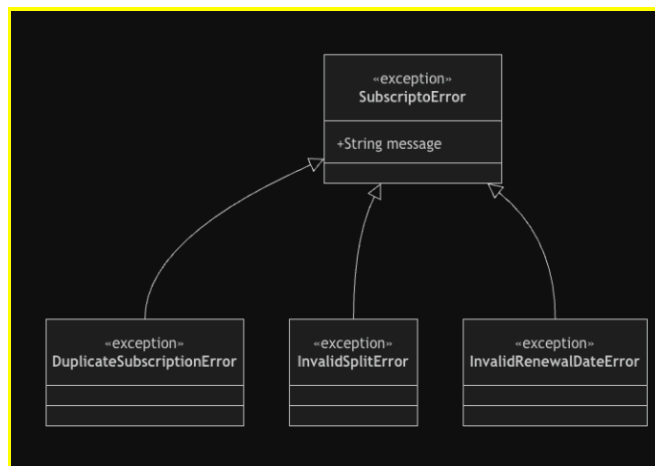
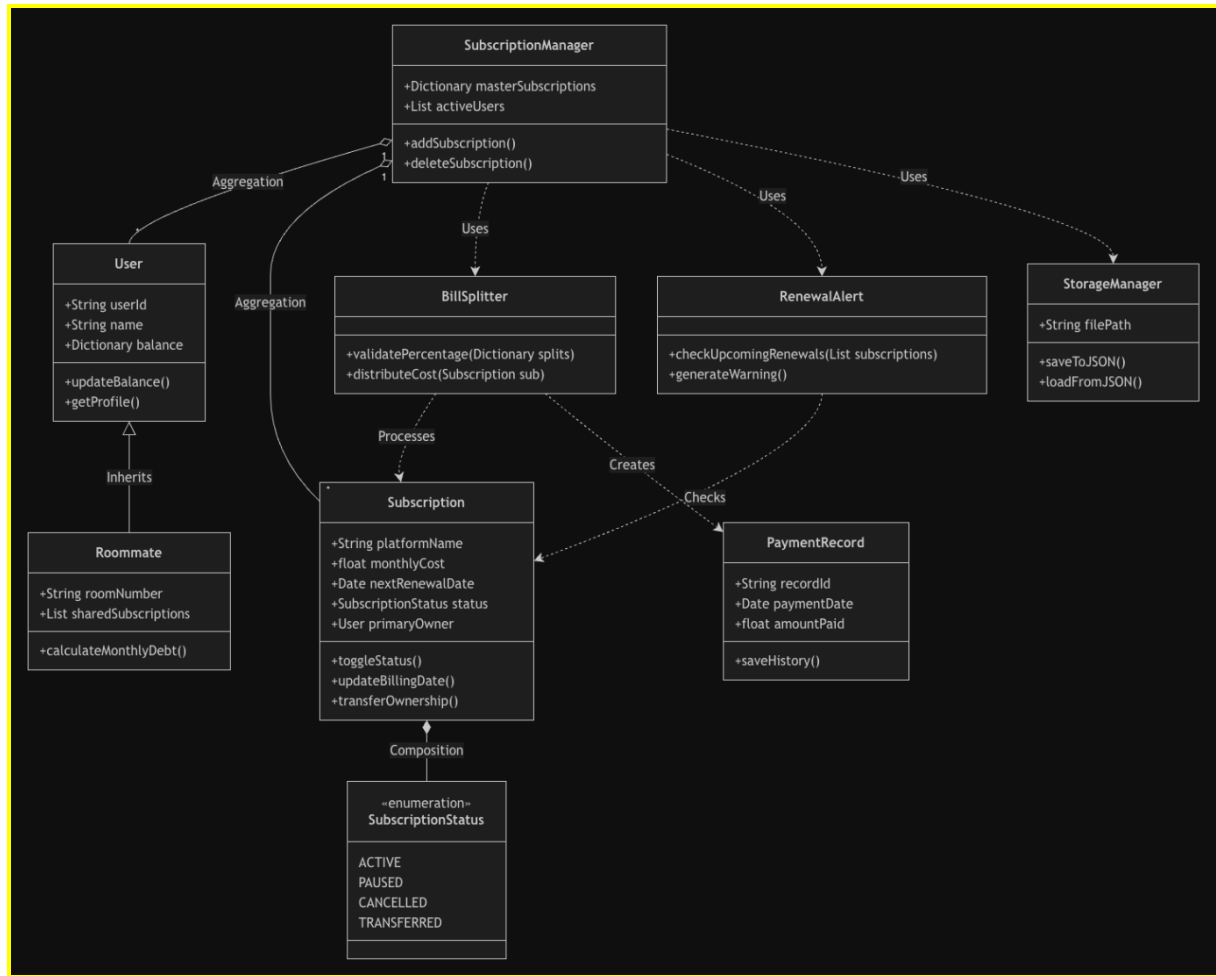
The Subscripto system utilizes a robust and structured object-oriented architecture to manage users, track joint household expenses, and execute automated lifecycle operations. The foundational class of the system is the User class, which encapsulates essential identity and financial tracking for any individual on the platform, containing a unique user identification string, user name, and an internal balance registry. To efficiently model co-living dynamics, the Roommate class inherits directly from the User class, introducing specialized attributes such as a designated room number and a list of specific active subscriptions they are responsible for splitting. Core subscription logic is tightly encapsulated within the Subscription class, which tracks service platform names, billing dates, monthly costs, and the primary account owner. To model various distinct lifecycles without relying on loose strings, the SubscriptionStatus class formally represents the operational state of a service (active, paused, cancelled, or transferred). System-wide workflows are orchestrated by a central SubscriptionManager class acting as the master controller, which coordinates the user registry and subscription records. To fulfill advanced financial and behavioral requirements, specialized structural components have been introduced. Financial distribution and validation are handled cleanly by the BillSplitter class, while temporal filtering and proximity warnings are evaluated by the RenewalAlert class. Persistence is delegated entirely to the StorageManager class, which safely reads and writes data through JSON or CSV serialization. Ledger histories are permanently captured via the PaymentRecord class. To ensure architectural resilience and robust error boundaries, a custom base exception

SubscriptoError is introduced, from which three domain-specific exceptions inherit: DuplicateSubscriptionError, InvalidSplitError, and InvalidRenewalDateError.

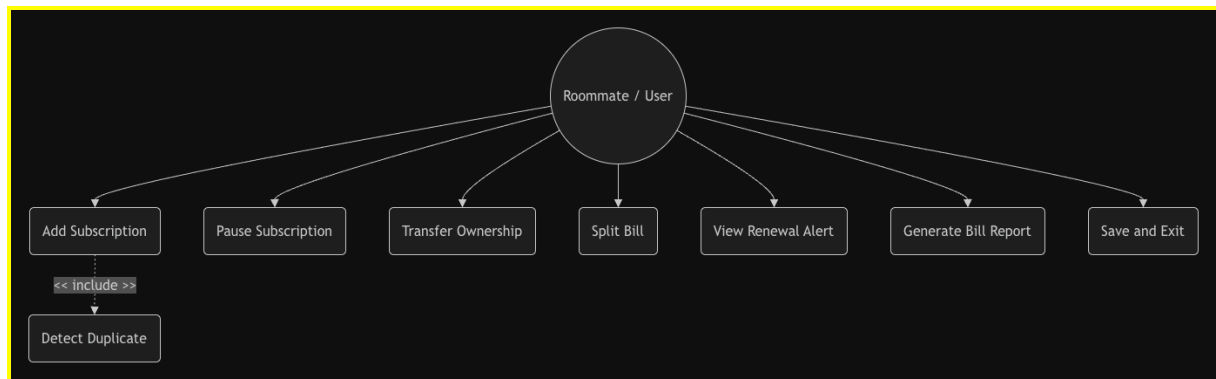
Class / Component	Purpose
User	Represents a basic registered application user with identity and generic balance attributes.
Roommate	Inherits from User; handles household-specific attributes like room numbers and personal shared debt totals.
Subscription	Models a recurring digital service, storing its platform name, cost, primary owner, and renewal schedule.
SubscriptionStatus	An enumeration or representation of the subscription's state (Active, Paused, Cancelled, Transferred).
SubscriptionManager	Acts as the main controller handling user registries, duplicate filtering, and routing high-level workflows.
Billsplitter	Responsible for validating splitting mathematical logic and calculating exact household debt distributions.
RenewalAlert	Periodically scans data models to filter and trigger notifications for renewals due within the next 24 hours.
PaymentRecord	Logs persistent individual billing results, historic split outputs, and payment summaries.
StorageManager	Manages local disk I/O, persisting and fetching structured application states via JSON or CSV formats.
SubscriptoError	Custom base exception class for the application to provide clean exception handling boundaries.
DuplicateSubscriptionError	Inherits from SubscriptoError; raised immediately when a platform registration conflict is detected.

InvalidSplitError	Inherits from SubscriptoError; thrown if cumulative household bill splitting percentages do not equal exactly 100%.
InvalidRenewalDateError	Inherits from SubscriptoError; raised when subscription entry contains missing, corrupted, or invalid dates.

UML class diagram



Use Case Diagram



Data Structures and Algorithms Plan

The software relies on a combination of built-in Python data structures to organize records and ensure swift retrieval. The master collection of subscriptions inside the manager class will be stored in a standard Python dictionary where the keys are the unique platform names and the values are the corresponding Subscription objects. This dictionary structure allows the system to implement an efficient duplicate detection algorithm because it can instantly check if a platform key already exists before allowing a user to add a new service. For tracking the chronological payment dates, the system will utilize a sorted list structure that acts as a custom queue to arrange upcoming payments by their calendar dates.

```
subscriptions_by_platform = {
    "Netflix": Subscription(...),
    "Spotify": Subscription(...)
}
```

```
split_percentages = {
    "user_01": 50,
    "user_02": 25,
    "user_03": 25
}
```

The core algorithmic logic of the application focuses on financial distribution and temporal filtering. The bill splitting algorithm iterates through the shared users associated with a subscription and reads their specific percentage attributes to divide the cost. This algorithm

contains validation logic that sums the percentages and utilizes exception handling to catch scenarios where the total split does not equal exactly one hundred percent. Another critical filtering algorithm runs every time the system dashboard loads by comparing the current date against the renewal dates stored in the subscription dictionary. This algorithm filters out all items except those where the difference between the target date and the current date is exactly twenty four hours, triggering a console alert for the user.

```
if 0 <= (renewal_date - today).days <= 1: show renewal alert
```

Expected Features or Sample Output

The final application will deliver a text-based interactive command menu that allows household members to manage their joint expenses smoothly without needing a complex graphical interface. Users will be greeted with a main menu showing clear options to view the dashboard, add a new subscription, pause an existing service, or view the current shared bill breakdown.

Subscripto Main Menu
1. Add Subscription
2. View Dashboard
3. Pause Subscription
4. Transfer Ownership
5. Generate Bill Split Report
6. View Renewal Alerts
7. Save and Exit

When the user chooses to display the main dashboard, the program will print a formatted text table showing all tracked services, their monthly costs, their current operational statuses, and the roommates involved. If a subscription renewal is happening the next day, the system will

print a prominent warning message at the top of the screen stating that a specific payment is due in exactly twenty four hours. If a user attempts to add a subscription for a platform that is already running inside the house, the system will output an error message stating that a duplicate active subscription was detected for that platform and the operation was aborted. Finally, choosing the bill splitting report option will generate a clear text printout showing exactly how much money each roommate owes to the primary account owners for that billing cycle.

Test Case	Expected Result
Add new subscription	Subscription is saved.
Add duplicate platform	DuplicateSubscriptionError is shown.
Split bill with total 100%	Amounts are calculated correctly.
Split bill with total below/above 100%	InvalidSplitError is raised.
Renewal due within next 24 hours	Alert is displayed.
Invalid renewal date	InvalidRenewalDateError is raised.
Transfer ownership	Owner changes and bill report updates.
Save and reload JSON file	Users and subscriptions are restored.

Conclusion

The development of Subscripto will successfully address the growing modern challenge of managing fragmented digital expenses and shared household financial tracking. By structuring the software around rigorous object-oriented programming methodologies, the application will provide a stable, scalable, and intuitive local environment for single or collaborative users. Features such as proactive twenty four hour renewal alerts, proactive duplicate account detection, and mathematically validated bill splitting logic demonstrate how practical everyday dilemmas can be elegantly resolved through software engineering.

Ultimately, this project will not only deliver a functional budget optimization tool for shared households but will also serve as a comprehensive practical application of core Python design concepts including inheritance, encapsulation, custom data management, and defensive error handling.